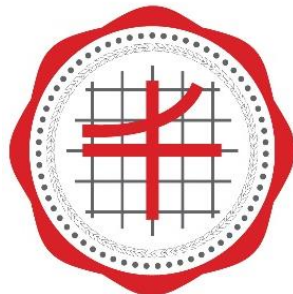


CPE 121 Mobile Application Developments

บทที่ 2

ตัวแปร ชนิดข้อมูล นิพจน์ และตัวดำเนินการ (Variable, Data Type, Expression and Operator)



Faculty of Engineering

SRINAKHARINWIROT UNIVERSITY

Pramual Choorat, Ph.D.

Department of Computer Engineering

ตัวแปร

ตัวแปร (Variable) คือ ชื่อหรือสัญลักษณ์ที่กำหนดขึ้น เพื่อการอ้างอิงตำแหน่งหน่วยความจำที่ใช้เก็บข้อมูล สามารถเก็บข้อมูลที่มีชนิดของข้อมูลแตกต่างกันได้และสามารถเปลี่ยนแปลงค่าข้อมูลได้ตลอดเวลา



กฎการตั้งชื่อตัวแปรในภาษา Java

- ตัวแปรนั้นต้องขึ้นต้นด้วยตัวอักษร
ห้ามตั้งชื่อตัวแปรขึ้นต้นด้วยตัวเลข, เครื่องหมาย, สัญลักษณ์อื่นใด
- ถัดจากตัวอักษรแรกของตัวแปรจะตามด้วยตัวอักษร หรือตัวเลข หรือเครื่องหมาย \$ หรือเครื่องหมาย _ ก็ได้
ห้ามเป็นเครื่องหมายอื่นๆ และห้ามเว้นช่องว่าง
- ตัวแปรในภาษา Java เป็น Case Sensitive นั่นคือ
การตั้งชื่อด้วยภาษาอังกฤษตัวพิมพ์เล็กจะแตกต่างจากการตั้งชื่อด้วยภาษาอังกฤษตัวพิมพ์ใหญ่
- ห้ามตั้งชื่อตัวแปรที่ตรงกับคีย์เวิร์ด (Keyword) คำสงวน (Reserved Word) ในภาษา Java



คำสงวน (Reserved Word) ในภาษา Java

abstract	do	if	private	this
assert	double	implements	protected	throw
boolean	else	import	public	throws
break	enum	instanceof	return	transient
byte	extends	int	short	true
case	false	interface	static	try
catch	final	long	strictfp	void
char	finally	native	super	volatile
class	float	new	switch	while
const	for	null	synchronized	continue
default	goto	package		

ตัวอย่างการตั้งชื่อตัวแปร

ชื่อตัวแปร	ถูกหรือผิด
X	✓
dayofWeek	✓
3dGraph	✗
Data1	✓
while	✗
Week day	✗

การประกาศตัวแปร (Variable Declaration)

- รูปแบบการประกาศตัวแปร

dataType **VarName** ; หรือ **dataType** **VarName** = **Value**;

โดยที่

dataType
VarName
Value

เป็นชนิดข้อมูลของตัวแปรที่ต้องการ
เป็นชื่อตัวแปรโดยตั้งตามกฎการตั้งชื่อ
เป็นค่าของตัวแปร



การประกาศตัวแปร (Variable Declaration)

ตัวอย่าง เช่น

String name;	//ประกาศตัวแปร name แทนข้อมูลชื่อ เป็นชนิดข้อความ
float score;	//ประกาศตัวแปร score แทนข้อมูลคะแนนสอบ เป็นชนิด เลขทศนิยม
char grade;	//ประกาศตัวแปร grade แทนข้อมูลเกรด เป็นชนิดข้อมูล ตัวอักษร

การประกาศตัวแปรพร้อมกับการกำหนดค่าเริ่มต้น

String name = "Boy";	//ประกาศตัวแปร name มีค่าเริ่มต้นเท่ากับ Boy
float score = 20.0f;	//ประกาศตัวแปร score มีค่าเริ่มต้นเท่ากับ 20.0
char grade = 'A';	//ประกาศตัวแปร grade มีค่าเริ่มต้นเท่ากับ A



ชนิดข้อมูล (Data Type)

- แบ่งเป็น 3 กลุ่ม ตามรูปแบบการใช้งาน ได้แก่
 - **Primitive Data Types** ชนิดของข้อมูลพื้นฐานของภาษา
 - Integer (จำนวนเต็ม)
 - Floating point (เลขทศนิยม)
 - Textual (อักขระ)
 - Logical (ตรรกะ/ค่าความจริง)
 - **Class Types** ชนิดของข้อมูลที่ถ่ายทอดมาจากคลาสหรือ Interface
 - **Array Types** ชนิดของข้อมูลที่เป็นอาร์เรย์ หรือข้อมูลเป็นชุดๆ



ข้อมูลชนิดจำนวนเต็ม (Integer)

- เป็นข้อมูลตัวเลขที่**ไม่มีค่าเป็นทศนิยม**
- ใช้เป็นค่าคงที่ ตัวแปร และนิพจน์ได้

ชนิดข้อมูล	ขนาด (ไบต์)	ช่วงการเก็บข้อมูล (ค่าที่เป็นไปได้)
byte	1	-128 ถึง +127
short	2	-32,768 ถึง +32,767
int	4	-2,147,483,648 ถึง +2,147,483,647
long	8	-9,223,372,036,854,775,808 ถึง +9,223,372,036,854,775,807



ข้อมูลชนิดจำนวนเต็ม (Integer)

- ข้อมูลเลขจำนวนเต็มชนิด **long** จะใช้อักษร **L** หรือ **l** ไว้ท้ายค่าตัวเลข เช่น

2147483648**L** แสดงว่าข้อมูลเลขนี้จะเก็บในตัวแปรชนิด **long**

- ในกรณีที่ไม่มีตัวอักษร **L** หรือ **l** ต่อท้ายค่าตัวเลข คอมไพเลอร์จะรู้ว่าเลขดังกล่าวเป็นเลขจำนวนเต็มชนิด **int** เช่น

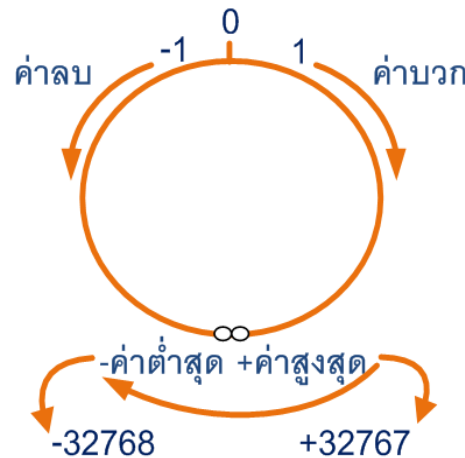
long D = 2147483648;

คอมไพเลอร์จะมอง **2147483648** เป็นเลขจำนวนเต็มชนิด **int** ซึ่งมีค่าเกินช่วงการเก็บข้อมูลของชนิด **int** จึงทำให้เกิด compiler error: **integer number too large: 2147483648**



ปัญหา overflow ของข้อมูล

- การกำหนดตัวแปรเพื่อเก็บข้อมูลเลขจำนวนเต็มที่มีมากกว่า 32767 เป็นข้อมูลชนิด **short integer** ทำให้เกิดปัญหา **overflow** ของข้อมูล



short เก็บข้อมูลขนาด **2 ไบต์**
ช่วงข้อมูลคือ -32,768 ถึง +32,767

ปัญหา overflow ของข้อมูล

```
public class IntegerRange {  
    public static void main(String[] args) {  
        short max = 32767;  
        System.out.println(max);  
        max++;  
        System.out.println(max);  
        max++;  
        System.out.println(max);  
    }  
}
```

> run IntegerRange

32767

-32768

-32767



ข้อมูลชนิดเลขทศนิยม (Floating Point)

- เป็นข้อมูลตัวเลขที่มีค่าเป็นทศนิยม
- สามารถเขียนให้อยู่ในรูปตัวเลขยกกำลังได้

ชนิดข้อมูล	ขนาด (ไบต์)	ช่วงการเก็บข้อมูล (ค่าที่เป็นไปได้)
float	4	-3.40292347E+38 ถึง +3.40292347E+38
double	8	-1.79769313486231570E+308 ถึง +1.79769313486231570E+308



ข้อมูลชนิดเลขทศนิยม (Floating Point)

- ข้อมูลชนิดตัวทศนิยมจะใช้อักษร F หรือ f ไว้ท้ายค่าตัวเลขทศนิยม เช่น 0.07**F** แสดงว่าข้อมูลเลขทศนิยมนี้จะเก็บในตัวแปรชนิด **float**
- ใช้ตัวอักษร D หรือ d ไว้ท้ายค่าตัวเลขทศนิยม เช่น 3.14**D** แสดงว่าข้อมูลเลขทศนิยมนี้จะเก็บในตัวแปรชนิด **double**
- สามารถกำหนดค่าตัวเลขจำนวนเต็มเพื่อเก็บในตัวแปรชนิด float หรือ double ได้ โดยไม่ต้องมีใช้ตัวอักษรต่อท้ายค่าตัวเลข เช่น
`float A = 200;`
`double B = 200;`



ข้อมูลชนิดตัวอักษร (Character)

- เป็นข้อมูลที่มีความยาวหนึ่งตัวอักษร
- ใช้พื้นที่หน่วยความจำ 2 ไบต์ต่อ 1 ตัวอักษร
- เก็บข้อมูลได้ถึง 65,536 ตัวอักษร
- ข้อมูลในกลุ่มนี้ ได้แก่
 - ตัวอักษร (A ถึง Z, a ถึง z)
 - ตัวเลข (0 ถึง 9)
 - สัญลักษณ์พิเศษต่าง ๆ เช่น + - = \$ * ฯลฯ



ข้อมูลชนิดตัวอักษร (Character)

- การใช้งานตัวอักษรต้องอยู่ภายในเครื่องหมาย **single quote** เช่น 'J', 'm', '9' เป็นต้น
- สามารถเรียงลำดับตัวอักษรตามค่าของรหัสแอสกี (ASCII: American Standard Code for Information Interchange) ได้ ตัวอย่างเช่น

อักขระ	รหัสแอสกี
'0'	48
'1'	49
'A'	65
'B'	66
'a'	97
'b'	98



โปรแกรมแสดงตัวเลขรหัสแอสกีและเลขฐานสอง

```
public class AsciiTest {  
    public static void main(String[] args) {  
        char c = 'A';  
        System.out.println("Ascii of " + c + " is " + (int)c);  
        System.out.println("Convert to Binary Number = "+ Integer.toBinaryString((int)c));  
    }  
}
```

> run AsciiTest

Ascii of A is 65

Convert to Binary Number = 1000001



ข้อมูลชนิดค่าความจริง (Boolean)

- เป็นข้อมูลที่มีค่าความจริงเป็นจริงหรือเป็นเท็จเท่านั้น
- แทนค่าด้วย true หรือ false
- ไม่ใช่เลขจำนวนเต็มหรือค่าอื่นๆ แทนค่าความจริง

```
public class BooleanTest {  
    public static void main(String[] args) {  
        boolean x = false ;    boolean y = true ;    boolean z = x || y ;  
        System.out.println ("give x = false, y = true and z = x || y then");  
        System.out.println ("!x = " + !x);  
        System.out.println ("x && y = " + (x && y));  
        System.out.println ("z = " + z);  
    }  
}
```

> run BooleanTest

give x = false, y = true and z = x || y then

!x = true

x && y = false

z = true

ข้อมูลชนิดข้อความ (String Type)

- ใช้ตัวแปรเก็บข้อมูลชนิดข้อความในรูปแบบของคลาส
- การใช้งานข้อความต้องอยู่ภายในเครื่องหมาย **double quote** เช่น **"JAVA"**, **"Program"** เป็นต้น ตัวอย่างเช่น

```
String strProvince = "Bangkok";
```

```
String strMonth = "May";
```



ค่าคงที่ (Constant)

- ค่าคงที่ ทำหน้าที่เก็บข้อมูลที่ต้องการกำหนดไว้ไม่ให้มีการเปลี่ยนแปลงตลอดเวลา เช่น ค่า PI เป็นต้น
- การประกาศค่าคงที่เหมือนกับการประกาศตัวแปรที่มีการกำหนดค่าเพียงแต่จะมีการใช้คีย์เวิร์ด **final** นำหน้า
- รูปแบบการใช้งาน

final dataType VarName ; หรือ **final** dataType VarName = Value;

โดยที่

dataType
VarName
Value

เป็นชนิดข้อมูลของค่าคงที่ที่ต้องการ
เป็นชื่อตัวแปรโดยตั้งตามกฎการตั้งชื่อ
เป็นค่าของค่าคงที่

เลขจำนวนเต็ม

- **int** เป็น default
- **long** ต้องระบุ l หรือ L หลังเลข

เลขทศนิยม

- **double** เป็น default
- **float** ต้องระบุ f หรือ F หลังเลข

ค่าคงที่ (Constant): โปรแกรมการใช้งานค่าคงที่

```
public class variable {  
    public static void main(String[] args) {  
        char c = 'Y';    int i = 1;    double d = 2.5;  
        float f = 1.2f;    boolean b = true;  
        final double SALES_TAX_RATE = 0.07d;  
        String s = "Hello";  
        System.out.println("c = " + c);  
        System.out.println("i =" + i);  
        System.out.println("d =" + d);  
        System.out.println("f =" + f);  
        System.out.println("b =" + b);  
        System.out.println("SALES_TAX_RATE = " + SALES_TAX_RATE);  
        System.out.println("s =" + s);  
    }  
}
```

> run BooleanTest

c = Y

i =1

d = 2.5

f = 1.2

b = true

SALES_TAX_RATE =
0.07

s = Hello

ตัวดำเนินการ (Operator)

- ทำหน้าที่จัดการหรือดำเนินการใด ๆ ระหว่างตัวแปร หรือ ตัวถูกกระทำ (Operand)

- Operand Operator Operand → Expression

- นิพจน์ (Expression) หมายถึง

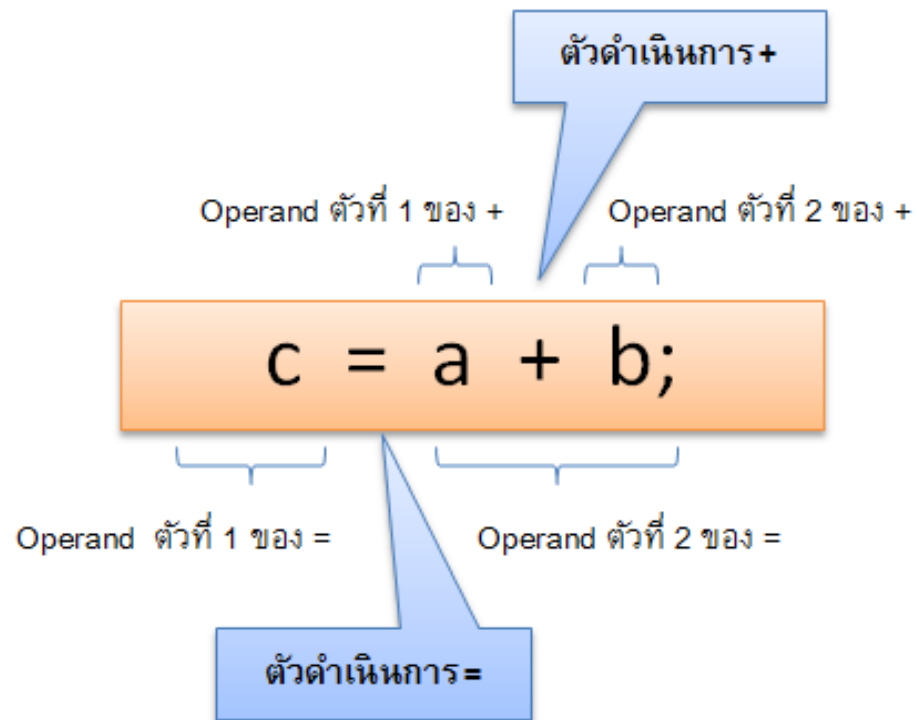
- ข้อความหรือประโยคที่เขียนอยู่ในรูปสัญลักษณ์ โดยนำข้อมูล, ตัวแปร, ฟังก์ชันหรือค่าคงที่ มาสัมพันธ์กับตัวดำเนินการ (Operator) อย่างใดอย่างหนึ่ง
- นิพจน์ 1 นิพจน์ จะต้องมิตัวถูกกระทำ (Operand) และตัวดำเนินการ (Operator) อย่างน้อย 1 ตัว



ตัวดำเนินการ (Operator) : ตัวอย่างนิพจน์

$X = Y$

$A = B - (C - 7)$



ตัวดำเนินการกำหนดค่า (Assignment Operators)

- ใช้สำหรับกำหนดค่าข้อมูลให้กับตัวแปรทางด้านซ้ายของตัวดำเนินการ =

ตัวดำเนินการ	คำอธิบาย	ตัวอย่าง
=	กำหนดค่า	$x = y$ คือ นำค่า y มาใส่ใน x
+=	เพิ่มค่าแล้วให้ค่าใหม่	$x += y$ คือ $x = x + y$ หรือนำ $x + y$ แล้วให้ค่าใหม่กับ x
-=	ลดค่าแล้วให้ค่าใหม่	$x -= y$ คือ $x = x - y$ หรือนำ $x - y$ แล้วให้ค่าใหม่กับ x
*=	คูณแล้วให้ค่าใหม่	$x *= y$ คือ $x = x * y$ หรือนำ $x * y$ แล้วให้ค่าใหม่กับ x
/=	หารแล้วให้ค่าใหม่	$x /= y$ คือ $x = x / y$ หรือนำ x / y แล้วให้ค่าใหม่กับ x

ตัวอย่างการทำงานของตัวดำเนินการกำหนดค่า

- กำหนดให้ตัวแปร A เป็นชนิดเลขจำนวนเต็ม และ $A = 25$

นิพจน์	การแทนค่านิพจน์	ค่าของนิพจน์
$A += 5$	$A = A + 5$	30
$A -= 25$	$A = A - 25$	0
$A *= 10$	$A = A * 10$	250
$A /= 25$	$A = A / 25$	1
$A \% = 5$	$A = A \% 5$	0

ตัวดำเนินการคณิตศาสตร์ (Arithmetic operators)

ตัวดำเนินการ	คำอธิบาย	ตัวอย่าง
+	บวก	$x + y$ ผลลัพธ์คือ x บวก y
-	ลบ	$x - y$ ผลลัพธ์คือ x ลบ y
*	คูณ	$x * y$ ผลลัพธ์คือ x คูณ y
/	หาร	x / y ผลลัพธ์คือ x หาร y
%	เศษจากการหาร	$x \% y$ ผลลัพธ์คือ เศษที่ได้จาก x หาร y (เช่น $13 \% 5$ เท่ากับ 3)

ผลการทำงานของตัวดำเนินการคณิตศาสตร์

- ผลการทำงานของตัวดำเนินการ $+$, $-$, $*$ และ $/$ ที่มีตัวแปรเป็นข้อมูลชนิดเลขทศนิยม จะได้เป็นค่าทศนิยมเสมอ
- ผลการทำงานของตัวดำเนินการ $+$, $-$, $*$ และ $/$ ที่มีตัวแปรเป็นข้อมูลชนิดเลขจำนวนเต็ม จะได้เป็นค่าจำนวนเต็มเสมอ โดยที่ผลลัพธ์จากการหาร จะได้ค่าจำนวนเต็มที่ปัดทศนิยมทิ้ง
- ผลการทำงานของตัวดำเนินการ $\%$ จะได้เป็นค่าจำนวนเต็มเสมอ



ตัวอย่างการทำงานของตัวดำเนินการคณิตศาสตร์

- กำหนดให้ตัวแปร A, B และ C เป็นชนิดเลขจำนวนเต็ม และ $A = 11$ และ $B = 5$

นิพจน์	A	B	C
$C = A + B$	11	5	16
$C = A - B$	11	5	6
$C = A * B$	11	5	55
$C = A / B$	11	5	2
$C = A \% B$	11	5	1



ตัวอย่างการทำงานของตัวดำเนินการคณิตศาสตร์

- กำหนดให้ตัวแปร A, B และ C เป็นชนิดเลขจำนวนทศนิยม และ $A = 10.5$ และ $B = 2.5$

นิพจน์	A	B	C
$C = A + B$	10.5	2.5	13.0
$C = A - B$	10.5	2.5	8.0
$C = A * B$	10.5	2.5	26.25
$C = A / B$	10.5	2.5	4.2
$C = A + B$	10.5	2.5	13.0



ตัวดำเนินการเปรียบเทียบ(Comparison Operators)

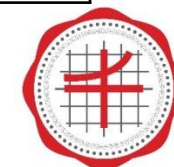
- ใช้สำหรับเปรียบเทียบข้อมูลระหว่างตัวถูกกระทำทางด้านซ้ายและด้านขวาของตัวดำเนินการ ได้ผลที่มีค่าเป็น **จริง (true)** หรือ **เท็จ (false)**
- ใช้ในการตัดสินใจกำหนดทิศทางของการทำงานของโปรแกรม

ตัวดำเนินการ	คำอธิบาย	ตัวอย่าง
==	เท่ากันหรือไม่	$x == y$ จะเป็นจริงเมื่อ x เท่ากับ y
!=	ไม่เท่ากันหรือไม่	$x != y$ จะเป็นจริงเมื่อ x ไม่เท่ากับ y
<	น้อยกว่า	$x < y$ จะเป็นจริงเมื่อ x น้อยกว่า y
<=	น้อยกว่าหรือเท่ากับ	$x <= y$ จะเป็นจริงเมื่อ x น้อยกว่าหรือเท่ากับ y
>	มากกว่า	$x > y$ จะเป็นจริงเมื่อ x มากกว่า y
>=	มากกว่าหรือเท่ากับ	$x >= y$ จะเป็นจริงเมื่อ x มากกว่าหรือเท่ากับ b

ตัวอย่างการทำงานของตัวดำเนินการเปรียบเทียบ

- กำหนดให้ตัวแปร A, B เป็นชนิดเลขจำนวนทศนิยม ตัวแปร C, D เป็นข้อมูลชนิดตัวอักขระ และ $A = -3$, $B = 5$, $C = 't'$ และ $D = 's'$

นิพจน์	A	B
$A \leq 10$	$-3 \leq 5$	true
$A+B > 0$	$(-3)+5 < 0$	false
$A-(-3) = B-5$	$(-3)-(-3) = 5-5$	true
$C == 't'$	$'t' == 't'$	true
$C == 'T'$	$'t' == 'T'$	false
$C < D$	$'t' < 's'$	false
$D > 'a'$	$'s' > 'a'$	true



ตัวดำเนินการตรรกศาสตร์ (Logical Operators)

- ใช้สำหรับกำหนดเงื่อนไขให้กับข้อมูลที่จะนำมาประมวลผลในทางตรรกศาสตร์ ในกรณีที่มีเงื่อนไขมากกว่า 1 เงื่อนไข ได้ผลที่มีค่าเป็น **จริง (true)** หรือ **เท็จ (false)**
- ใช้ในการตัดสินใจกำหนดทิศทางของการทำงานของโปรแกรม เช่นกัน

ตัวดำเนินการ	ความหมาย	ตัวอย่าง	คำอธิบาย
&&	and	x && y	มีค่าเป็นจริง ก็ต่อเมื่อ x และ y มีค่าเป็นจริง
	or	x y	มีค่าเป็นจริง ก็ต่อเมื่อ x หรือ y หรือ ทั้ง x และ y มีค่าเป็นจริง
!	not	! y	มีค่าเป็นจริง ก็ต่อเมื่อ x มีค่าเป็นเท็จ

ผลการทำงานของตัวดำเนินการตรรกศาสตร์

- เมื่อนำมาเปรียบเทียบค่าความจริงของนิพจน์ โดยกำหนดให้ x และ y เป็นตัวถูกดำเนินการทางตรรกศาสตร์ จะได้ผลการทำงานดังตาราง

x	y	x && y	x y	!x	!y
true	true	true	true	false	false
true	false	false	true	false	true
false	true	false	true	true	false
false	false	false	false	true	true



ตัวอย่างการทำงานของตัวดำเนินการตรรกศาสตร์

- กำหนดให้ตัวแปร A เป็นชนิดเลขจำนวนทศนิยม และ ตัวแปร B เป็นข้อมูลชนิดตัวอักขระ และ $A = 10$ และ $B = 'X'$

นิพจน์	การแทนค่านิพจน์	ค่าของนิพจน์
$(A > 0) \ \&\& \ (A < 15)$	T && T	true
$(A > 0) \ \&\& \ (A < 10)$	T && F	false
$(A > 0) \ \ (A < 5)$	T F	true
$(A <> 5) \ \ (B == 'X')$	T T	true
$! (A != 10)$	! (F)	true
$! (B > 'c')$	! (F)	true



ตัวดำเนินการยูนารี (Unary Operators)

- ตัวดำเนินการที่มี operand (ตัวถูกกระทำ) เพียง 1 ตัว เช่น **การเพิ่มค่า/ลดค่า**

ตัวดำเนินการ	ความหมาย	ตัวอย่าง	การทำงาน	ข้อสังเกต
++	เพิ่มค่าหนึ่งค่าให้กับตัวแปร	$A = B++$	$A = B$ $B = B + 1$	กำหนดค่า ให้กับตัวแปร A ก่อน เพิ่มค่า ตัวแปร B
		$A = ++B$	$B = B + 1$ $A = B$	เพิ่มค่า ให้ตัวแปร B ก่อน กำหนดค่า ให้กับตัวแปร A
--	ลดค่าลงหนึ่งค่าให้กับตัวแปร	$A = B--$	$A = B$ $B = B - 1$	กำหนดค่า ให้กับตัวแปร A ก่อน ลดค่า ตัวแปร B
		$A = --B$	$B = B - 1$ $A = B$	ลดค่า ให้ตัวแปร B ก่อน กำหนดค่า ให้กับตัวแปร A



ตัวอย่างการทำงานของตัวดำเนินการยูนารี

- กำหนดให้ตัวแปร A, B เป็นชนิดเลขจำนวนเต็ม และ $A = 5$

นิพจน์	A	B
$B = A++$	6	5
$B = ++A$	6	6
$B = A-- + 4$	4	9
$B = ++A + 4$	6	10
$B = 5 * A--$	4	25

ตัวอย่างการทำงานของตัวดำเนินการยูนารี

```
public class unary {
    public static void main(String[] args) {
        int A=5, B;   B = A++;
        System.out.println("1. A = " + A + " B = " + B);
        A=5;   B = ++A;
        System.out.println("2. A = " + A + " B = " + B);
        A=5;   B = A-- + 4;
        System.out.println("3. A = " + A + " B = " + B);
        A=5;   B = ++A + 4;
        System.out.println("4. A = " + A + " B = " + B);
        A=5;   B = 5 * A--;
        System.out.println("5. A = " + A + " B = " + B);
    }
}
```

> run unary

1. A = 6 B = 5
2. A = 6 B = 6
3. A = 4 B = 9
4. A = 6 B = 10
5. A = 4 B = 25



โปรแกรมการทำงานของตัวดำเนินการยูนารี

```
public class increment {  
    public static void main(String[] args) {  
        int x, y;  
        x=10;  
        y=x++;  
        System.out.println("x = "+x+" y= "+y);  
        y=++x;  
        System.out.println("x = "+x+" y= "+y);  
    }  
}
```

ส่งค่า x ไปให้ตัวแปร y ก่อน แล้วจึงเพิ่มค่า x

เพิ่มค่า x แล้ว จึง ส่งค่า x ไปให้ตัวแปร y

x = 11 y= 10
x = 12 y= 12



ตัวดำเนินการเทอร์นารี (Ternary Operator)

- ตัวดำเนินการเทอร์นารีเป็นการเปรียบเทียบโดยใช้โอเปอเรนด์ทั้งหมด 3 ตัว โดยมีรูปแบบดังนี้

(boolean-expression) ? true-expression : false-expression

รูปแบบของ **Ternary Operator** จะมีความหมายดังนี้

- ถ้า boolean-expression เป็น **จริง** จะทำการคืนค่าหรือทำตามข้อกำหนดใน **true-expression**
- ถ้า boolean-expression เป็น **เท็จ** จะทำการคืนค่าหรือทำตามข้อกำหนดใน **false-expression**
- โดยทั้ง **true-expression** และใน **false-expression** นั้น จะต้องมีการคืนค่าผลลัพธ์กลับมา หรือมีค่าอย่างใดอย่างหนึ่งเกิดขึ้น เพื่อนำค่าไปกำหนดให้กับตัวแปรที่อยู่ด้านซ้ายมือของเครื่องหมาย =



ตัวอย่างการทำงานของตัวดำเนินการเทอร์นารี

```
public class Ternary {  
    public static void main(String[] args) {  
        boolean x = (1 > 0) ? true : false;  
        boolean y = (0 != -0.0) ? true : false;  
  
        int x = 20;  
        String s = (x % 2 == 0) ? "Even" : "Odd";  
  
        String login = "admin";  
        String psw = "1234";  
        String msg = ( login.isEmpty() || psw.isEmpty()) ? "Please enter..." : "Thanks";  
    }  
}
```


ตัวอย่างการทำงานของตัวดำเนินการเทอร์นารี

```
import java.util.Scanner;

public class ExTernary {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        int numOdd = 0;          int numEven = 0;          int dummy = 0;

        System.out.print("จำนวนที่ #1: ");

        int n = scanner.nextInt();

        boolean b = (n % 2 == 0) ? true : false;

        dummy = (b == true) ? numEven++ : numOdd++;

        System.out.print("จำนวนที่ #2: ");

        n = scanner.nextInt();

        b = (n % 2 == 0) ? true : false;

        dummy = (b == true) ? numEven++ : numOdd++;
```

ตัวอย่างการทำงานของตัวดำเนินการเทอร์นารี

```
System.out.print("จำนวนที่ #3: ");
```

```
n = scanner.nextInt();
```

```
b = (n % 2 == 0) ? true : false;
```

```
dummy = (b == true) ? numEven++ : numOdd++; }
```

```
System.out.printf("จำนวนทั้งหมดมีเลขคู่: %d และเลขคี่: %d\n", numEven, numOdd);
```

```
}
```

```
}
```



ตัวดำเนินการระดับบิต (Bitwise Operators)

- ตัวดำเนินการจะกระทำกับทุกๆ บิตของตัวถูกดำเนินการที่มีชนิดข้อมูลในตระกูลเลขจำนวนเต็ม (ได้แก่ byte, short, int และ long) หรือเป็นตัวอักษร

ตัวดำเนินการ	คำอธิบาย
<<	left shift หรือเลื่อนบิตมาทางซ้าย และเติมเลข 0 ทางขวา เท่ากับจำนวนบิตที่เลื่อนไป
>>>	Unsigned right shift หรือเลื่อนบิตมาทางขวา และเติมเลข 0 ทางซ้ายเท่ากับจำนวนบิตที่เลื่อนไป
>>	Signed right shift หรือเลื่อนบิตมาทางขวา โดยพิจารณาเครื่องหมาย +/- ของข้อมูล คือจะเติมเลข 0 ทางซ้ายเท่ากับจำนวนบิตที่เลื่อนไป เมื่อข้อมูลเป็นค่าบวก และจะเติมเลข 1 ทางซ้ายเท่ากับจำนวนบิตที่เลื่อนไปเมื่อข้อมูลเป็นค่าลบ
&	bitwise AND
	bitwise OR
^	bitwise XOR

ผลการทำงานของตัวดำเนินการระดับบิต

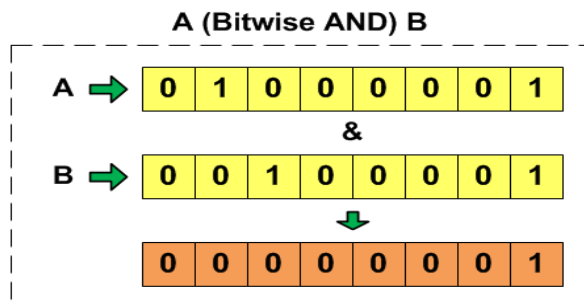
A	B	(Bitwise AND) A & B	(Bitwise OR) A B	(Bitwise XOR) A ^ B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

- การดำเนินการ **and** ผลลัพธ์จะเป็น **1** ถ้าตัวถูกดำเนินการตัวที่หนึ่งและตัวถูกดำเนินการตัวที่สองมีค่าเป็น **1** ทั้งคู่ นอกนั้นให้ผลลัพธ์เป็น **0**
- การดำเนินการ **or** ผลลัพธ์จะเป็น **1** ถ้าตัวถูกดำเนินการตัวใดตัวหนึ่งมีค่าเป็น **1** นอกนั้นให้ผลลัพธ์เป็น **0**
- การดำเนินการ **xor** ผลลัพธ์จะเป็น **1** ถ้าตัวถูกดำเนินการทั้งสองมีค่าต่างกัน และผลลัพธ์จะเป็น **0** เมื่อตัวถูกดำเนินการทั้งสองมีค่าเหมือนกัน

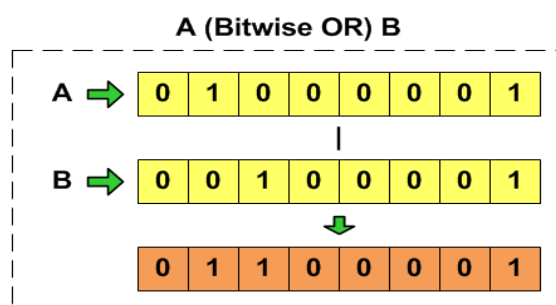
ตัวอย่างการทำงานของตัวดำเนินการระดับบิต

- กำหนดให้ขนาดเนื้อที่สำหรับเก็บค่าตัวแปร A และ B เท่ากับ 8 บิต
A = 01000001, B = 00100001 และ C = 11111000

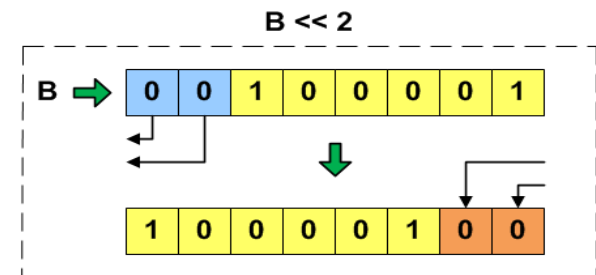
A & B = 00000001



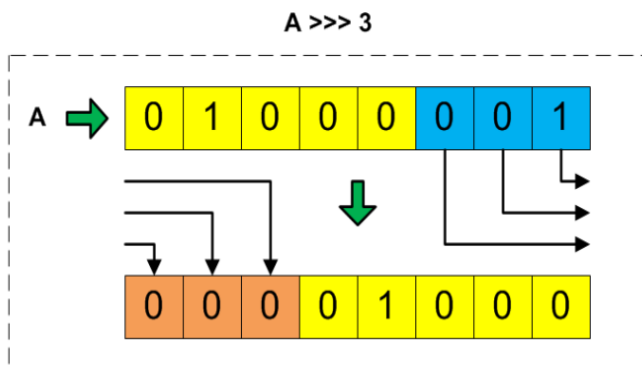
A | B = 01100001



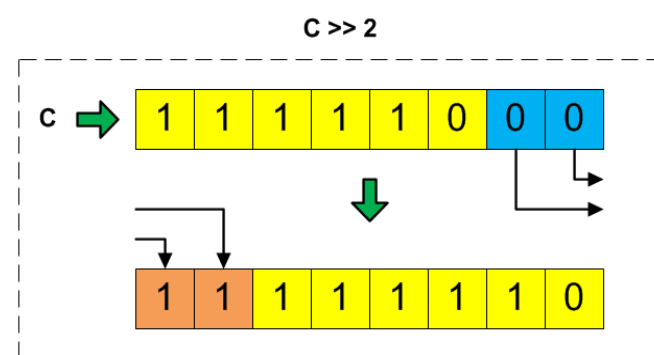
B << 2 = 10000100



A >>> 3 = 00001000



C >> 2 = 11111110



ตัวอย่างการทำงานของตัวดำเนินการระดับบิต

```
public class shift {  
    public static void main(String[] args) {  
        int a = 64;  
        int b = 33;  
        System.out.println(a & b);  
        System.out.println(a | b);  
        System.out.println(a ^ b);  
        System.out.println(a >> 3);  
        System.out.println(b << 2);  
    }  
}
```

a = 01000000
b = 00100001

> run shift

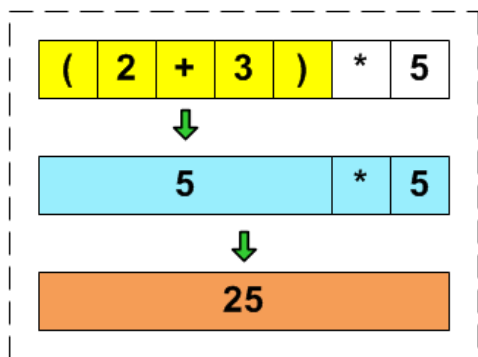
0
97
97
8
132

ลำดับความสำคัญของตัวดำเนินการ

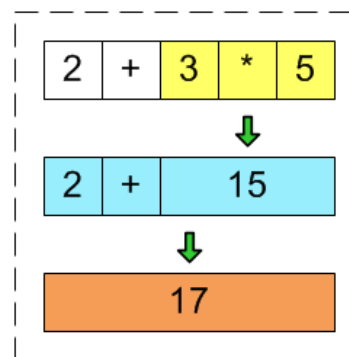
ลำดับที่	ตัวดำเนินการ	การกระทำ
1	(), []	ซ้ายไปขวา
2	++, --, !, ~	ขวาไปซ้าย
3	*, /, %	ซ้ายไปขวา
4	+, -	ซ้ายไปขวา
5	<<, >>, >>>	ซ้ายไปขวา
6	<, <=, >, >=	ซ้ายไปขวา
7	==, !=	ซ้ายไปขวา
8	&	ซ้ายไปขวา
9	^	ซ้ายไปขวา
10		ซ้ายไปขวา
11	&&	ซ้ายไปขวา
12		ซ้ายไปขวา
13	?:	ซ้ายไปขวา
14	=, +=, -=, *=, /=, %=, <<=, >>=, >>>=, &=, ^=, !=	ขวาไปซ้าย

ตัวอย่างการคำนวณของตัวดำเนินการ

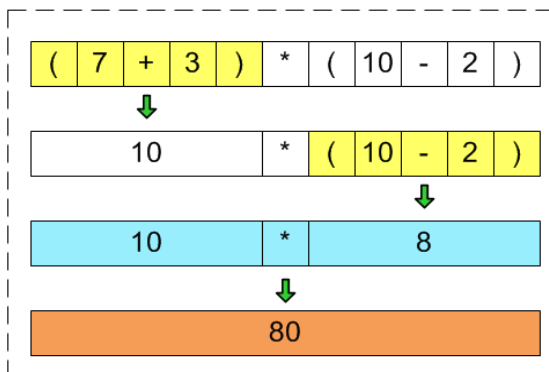
$$(2 + 3) * 5 = 25$$



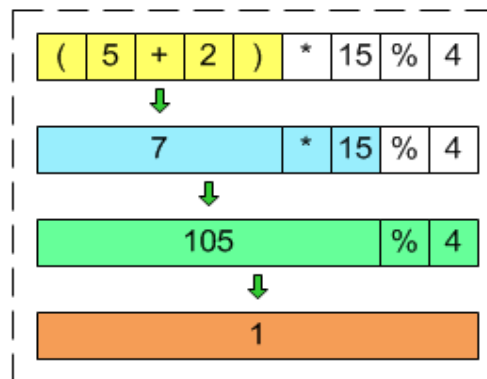
$$2 + 3 * 5$$



$$(7 + 3) * (10 - 2)$$



$$(5 + 2) * 15 \% 4$$



ผลการคำนวณของตัวดำเนินการทางคณิตศาสตร์

int x; x=5+2*4+15*2/7;	X=17
double x; x=5+2*4+15*2/7;	X=17.0
double x; x=5+2*4+15*2.0/7;	X=17.285714285714285
double x; x=5.0+2*4+15*2/7;	X=17.0



การแปลงชนิดของข้อมูล (Data Type Conversion)

▪ Implicit Type Conversion

- เป็นการแปลงชนิดของข้อมูลในกรณีที่ข้อมูลมีชนิดต่างกัน
- กระทำอัตโนมัติ โดยคอมไพเลอร์ ซึ่งมีเงื่อนไขดังนี้
 - ข้อมูลใด ๆ จะต้องมีชนิดของข้อมูลที่เข้ากันได้ (compatible type) คือ มีชนิดเป็นตัวเลขเหมือนกัน
 - เป็นการแปลงจากชนิดของข้อมูลที่มีขนาดเล็กไปเป็นชนิดของข้อมูลที่มีขนาดใหญ่กว่าเท่านั้น

การแปลงชนิดของข้อมูล (Data Type Conversion)

```
public class ImplicitTest {  
    public static void main(String[] args) {  
        float firstNo=5;  
        int secondNo=0;  
        System.out.println("firstNo = " + firstNo);  
        System.out.println("secondNo = " + secondNo);  
        firstNo = secondNo;  
        System.out.println("firstNo = " + firstNo);  
    }  
}
```

> run ImplicitTest

firstNo = 5.0

secondNo = 0

firstNo = 0.0



การแปลงชนิดของข้อมูล (Data Type Conversion)

- **Explicit Type Conversion (casting)** เป็นการแปลงโดยผู้เขียนโปรแกรม

`valNameResult = (dataType) valName`

โดยที่

<code>valNameResult</code>	เป็นชื่อตัวแปรที่รับค่าจากการแปลงข้อมูล ต้องมีชนิดข้อมูลเดียวกันกับ <code>dataType</code>
<code>dataType</code>	เป็นชนิดข้อมูลที่ต้องการแปลง
<code>valName</code>	เป็นชื่อตัวแปรหรือข้อมูลที่ต้องการแปลงชนิดข้อมูล

การแปลงชนิดของข้อมูล (Data Type Conversion)

```
public class ExplicitTest {  
    public static void main(String[] args) {  
        double tax1;    float vat1 = 0.07f;  
        int price1 = 250;  
        tax1 = (double)(vat1*price1);  
        System.out.println("TAX1 = " + tax1 + " BAHT");  
        int tax2;        float vat2 = 0.07f;  
        double price2 = 250;  
        tax2 = (int)(vat2*price2);  
        System.out.println("TAX2 = " + tax2 + " BAHT");  
    }  
}
```

> run ExplicitTest

TAX1 = 17.5 BAHT

TAX2 = 17 BAHT



การแปลงชนิดของข้อมูลผ่านคลาส Wrapper

- เป็นการแปลงค่าชนิดข้อมูลสายอักขระให้เป็นตัวเลขหรือเปลี่ยนค่าชนิดข้อมูลตัวเลขให้เป็นสายอักขระได้
- ใช้เมธอดในคลาส Wrapper ซึ่งเป็นคลาสที่สัมพันธ์กับข้อมูลชนิดพื้นฐาน (Primitive data type)
- เรียกผ่านชื่อคลาส Wrapper ที่สัมพันธ์กับชนิดข้อมูลที่ต้องการ

คลาส Wrapper	Primitive Data Type	เมธอดที่ใช้ เปลี่ยนเป็นตัวเลข	เมธอดที่ใช้ เปลี่ยนเป็นสายอักขระ
Integer	int	Integer.parseInt()	Integer.toString()
Float	float	Float.parseFloat()	Float.toString()
Double	double	Double.parseDouble()	Double.toString()



โปรแกรมการใช้งานเมธอดประเภท `parseType`

```
public class parseTypeTest {  
    public static void main(String[] args) {  
        String a = "25", b = "25.125", c="25.2525";  
        System.out.printf("a = %s\n", a);  
        int a1 = Integer.parseInt(a);  
        System.out.printf("a1 = %d\n", a1);  
        System.out.printf("b = %s\n", b);  
        float b1 = Float.parseFloat(b);  
        System.out.printf("b1 = %f\n", b1);  
        System.out.printf("c = %s\n", c);  
        double c1 = Double.parseDouble(c);  
        System.out.printf("c1 = %f\n", c1);  
    }  
}
```

> run `parseTypeTest`

```
a = 25  
a1 = 25  
b = 25.125  
b1 = 25.125000  
c = 25.2525  
c1 = 25.252500
```



โปรแกรมการใช้งานเมธอด toString()

```
public class toStringTest {  
    public static void main(String[] args) {  
        int a=25;    float b=25.25f;    double c=20.5d;  
        System.out.printf("a = %d\n", a);  
        String a1 = Integer.toString(a);  
        System.out.printf("a1 = %s\n", a1);  
        System.out.printf("b = %f\n", b);  
        String b1 = Float.toString(b);  
        System.out.printf("b1 = %s\n", b1);  
        System.out.printf("c = %f\n", c);  
        String c1 = Double.toString(c);  
        System.out.printf("c1 = %s\n", c1);  
    }  
}
```

> run toStringTest

```
a = 25  
a1 = 25  
b = 25.250000  
b1 = 25.25  
c = 20.500000  
c1 = 20.5
```



โปรแกรมการแปลงชนิดของข้อมูล

```
public class changeVar {  
    public static void main(String[] args) {  
        int a;  
        double b= 3;  
        int c = 5;  
        a =b+c;  
        System.out.println("a =" +a);  
    }  
}
```

Type mismatch: cannot
convert from double to int



เมธอดในคลาส Math

- เมธอด **pow()** เป็นเมธอดหาค่ายกกำลังของข้อมูล ซึ่งมีรูปแบบการใช้งานดังนี้

`Math.pow(x, y);`

โดยที่

x เป็นค่าตัวเลขที่ต้องการยกกำลัง

y เป็นค่าตัวเลขยกกำลัง

ผลที่ได้ คือ ค่าตัวเลขของ x ยกกำลังด้วย y (x^y)

- เมธอด **sqrt()** เป็นเมธอดหาค่ารากที่สองของข้อมูล ซึ่งมีรูปแบบการใช้งานดังนี้

`Math.sqrt(x);`

โดยที่

x เป็นค่าตัวเลขที่ต้องการถอดรากที่สอง

ผลที่ได้ คือ ค่ารากที่สองของ x ()



เมธอดในคลาส Math

- เมธอด **abs()** เป็นเมธอดหาค่าสัมบูรณ์ของข้อมูล มีรูปแบบ ดังนี้

`Math.abs(x);`

โดยที่

x เป็นค่าตัวเลขที่ต้องการหาค่าสัมบูรณ์
ผลที่ได้ คือ ค่าสัมบูรณ์ของ x

- เมธอด **round()** เป็นเมธอดปัดเศษทศนิยมเป็นจำนวนเต็ม มีรูปแบบ ดังนี้

`Math.round(x);`

โดยที่

x เป็นค่าตัวเลขที่ต้องการปัดเศษทศนิยม
ผลที่ได้ คือ ค่าตัวเลขใน 2 รูปแบบ คือ

- ถ้าตัวเลขหลังจุดทศนิยมมีค่ามากกว่าหรือเท่ากับ 5 จะปัดค่าตัวเลขหลังจุดทศนิยมขึ้น
- ถ้าตัวเลขหลังจุดทศนิยมมีค่าน้อยกว่า 5 จะตัดค่าตัวเลขหลังจุดทศนิยมทิ้ง



เมธอดในคลาส Math

- เมธอด **ceil()** เป็นเมธอดปัดเศษทศนิยมขึ้นเป็นจำนวนเต็ม มีรูปแบบ ดังนี้

`Math.ceil(x);`

โดยที่

x เป็นค่าตัวเลขที่ต้องการปัดเศษทศนิยมขึ้น

ผลที่ได้ คือ ค่าตัวเลขที่ปัดค่าหลังจุดทศนิยมขึ้นเป็นจำนวนเต็ม

- เมธอด **floor()** เป็นเมธอดปัดเศษทศนิยมลงเป็นจำนวนเต็ม มีรูปแบบ ดังนี้

`Math.floor(x);`

โดยที่

x เป็นค่าตัวเลขที่ต้องการปัดเศษทศนิยมทั้งไป

ผลที่ได้ คือ ค่าตัวเลขที่ปัดค่าตัวเลขหลังจุดทศนิยมลงเป็นจำนวนเต็ม

- เมธอด **PI()** เป็นเมธอดแสดงค่า $\pi = 3.141592653589793$

`Math.PI;`

ผลที่ได้ คือ ค่าตัวเลข π ซึ่งเท่ากับ 3.141592653589793

เมธอดในคลาส Math

- เมธอด **max()** เป็นเมธอดหาค่าสูงสุดจากข้อมูล 2 ตัว มีรูปแบบ ดังนี้

`Math.max(x, y);`

โดยที่

x เป็นค่าตัวเลขตัวที่ 1 ที่ต้องการหาค่าสูงสุด
y เป็นค่าตัวเลขตัวที่ 2 ที่ต้องการหาค่าสูงสุด
ผลที่ได้ คือ ค่าข้อมูลที่สูงที่สุดระหว่างค่า x กับค่า y

- เมธอด **min()** เป็นเมธอดหาค่าต่ำสุดจากข้อมูล 2 ตัว มีรูปแบบ ดังนี้

`Math.min(x, y);`

โดยที่

x เป็นค่าตัวเลขตัวที่ 1 ที่ต้องการหาค่าต่ำสุด
y เป็นค่าตัวเลขตัวที่ 2 ที่ต้องการหาค่าต่ำสุด
ผลที่ได้ คือ ค่าข้อมูลที่ต่ำที่สุดระหว่างค่า x กับค่า y



เมธอดในคลาส Math

- เมธอด **random()** เป็นเมธอดสุ่มตัวเลขเป็นจำนวนทศนิยมมีค่า 0-1 มีรูปแบบ ดังนี้

Math.random();

ผลที่ได้ คือ ค่าตัวเลขที่ได้จากการสุ่มเป็นจำนวนทศนิยมที่มีค่าระหว่าง 0-1

- เมธอด **log()** เป็นเมธอดหาค่า log ของข้อมูล มีรูปแบบ ดังนี้

Math.log(x);

โดยที่

x เป็นค่าตัวเลขที่ต้องการหาค่า log

ผลที่ได้ คือ ผลลัพธ์ของการหาค่า log ของ x ($\log x$)

- เมธอด **exp()** เป็นเมธอดหาค่า e ยกกำลังของข้อมูล มีรูปแบบ ดังนี้

Math.exp(x);

โดยที่

x เป็นค่าตัวเลขที่ต้องการหาค่า e ยกกำลัง

ผลที่ได้ คือ ผลลัพธ์ค่า exponential ของ x (e^x)



เมธอดในคลาส Math

- เมธอด **sin()** เป็นเมธอดหาค่า sine ของมุมที่กำหนด มีรูปแบบ ดังนี้

`Math.sin(x);`

โดยที่

x เป็นขนาดของมุมที่ต้องการหาค่า sin
ผลที่ได้ คือ ผลลัพธ์ค่า sine ของ x ($\sin(x)$)

- เมธอด **cos()** เป็นเมธอดหาค่า cosine ของมุมที่กำหนด มีรูปแบบ ดังนี้

`Math.cos(A);`

โดยที่

x เป็นขนาดของมุมที่ต้องการหาค่า cosine
ผลที่ได้ คือ ผลลัพธ์ค่า cosine ของ x ($\cos(x)$)



เมธอดในคลาส Math

- เมธอด **tan()** เป็นเมธอดหาค่า tangent ของมุมที่กำหนด มีรูปแบบ ดังนี้

`Math.tan(x);`

โดยที่

x เป็นขนาดของมุมที่ต้องการหาค่า tangent

ผลที่ได้ คือ ผลลัพธ์ค่า tangent ของ x ($\tan(x)$)

โปรแกรมแสดงการใช้เมธอดในคลาส Math

```
public class MathMethod {  
    public static void main(String[] args) {  
        System.out.println("1.Math.pow(3,2)= " + Math.pow(3,2));  
        System.out.println("2.Math.sqrt(9)= " + Math.sqrt(9));  
        System.out.println("3.Math.abs(-10)= "+Math.abs(-10));  
        System.out.println("4.Math.ceil(9.2)= " + Math.ceil(9.2));  
        System.out.println("5.Math.floor(9.2)= " + Math.floor(9.2));  
        System.out.println("6.Math.round(9.2)= " + Math.round(9.2));  
        System.out.println("7.Math.max(2,3)= " + Math.max(2,3));  
        System.out.println("8.Math.min(2,3)= " + Math.min(2,3));  
        System.out.println("9.Math.PI = " + Math.PI);  
        System.out.println("10.Math.random()= " + Math.random());  
        System.out.println("11.Math.log(10)= "+Math.log(10) );  
        System.out.println("12.Math.exp(1.0)= " + Math.exp(1.0));  
        System.out.println("13.Math.sin(0.0)= " + Math.sin(0.0));  
        System.out.println("14.Math.cos(0.0)= " + Math.cos(0.0));  
        System.out.println("15.Math.tan(0.0)= " + Math.tan(0.0));  
    }  
}
```

โปรแกรมแสดงการใช้เมธอดในคลาส Math

> run MathMethod

```
1.Math.pow(3,2)= 9.0
2.Math.sqrt(9)= 3.0
3.Math.abs(-10)= 10
4.Math.ceil(9.2)= 10.0
5.Math.floor(9.2)= 9.0
6.Math.round(9.2)= 9
7.Math.max(2,3)= 3
8.Math.min(2,3)= 2
9.Math.PI = 3.141592653589793
10.Math.random()= 0.5058925234813406
11.Math.log(10)= 2.302585092994046
12.Math.exp(1.0)= 2.718281828459045
13.Math.sin(0.0)= 0.0
14.Math.cos(0.0)= 1.0
15.Math.tan(0.0)= 0.0
```



เมธอดในคลาส Calendar

คลาส Calendar เป็นเมธอดที่ทำงานกับข้อมูลด้านวันที่และเวลา อยู่ในแพ็คเกจ java.util.Calendar ในการใช้งานผู้อ่านจะต้องสร้างออบเจกต์ของคลาส Calendar ก่อน จากนั้นเรียกใช้เมธอดที่ต้องการ

```
Calendar calenName = Calendar.getInstance( );  
System.out.println("Current DateTime is : " + calenName.getTime());
```

> run CalendarTest

Current DateTime is : Sat Jul 23 13:16:05 ICT 2016

โดยที่

calenName	เป็นชื่อออบเจกต์
getInstance	เป็นเมธอดสำหรับการสร้างออบเจกต์ปฏิทิน
getTime	เป็นเมธอดที่ให้ค่าวันที่และเวลาปัจจุบัน

เมธอด format ของคลาส SimpleDateFormat

คลาส **SimpleDateFormat** ใช้แสดงผลวันที่และเวลาในรูปแบบที่แตกต่างกัน อยู่ในแพ็คเกจ `java.text.SimpleDateFormat` มีรูปแบบ ดังนี้

```
Calendar calenName = Calendar.getInstance( );  
SimpleDateFormat df = new SimpleDateFormat("d/MM/yyyy HH:mm:ss");  
System.out.println("Current DateTime is : " + df.format(calenName.getTime()));
```

> run CalendarTest

Current DateTime is : 23/07/2559 13:17:41



รหัสแสดงผลวันเดือนปี และเวลา

รหัสแสดงผล	คำอธิบาย
yyyy	แสดงปีศักราช
MM	แสดงเดือนเป็นตัวเลข 2 หลัก
ww	แสดงสัปดาห์ของปี
WW	แสดงสัปดาห์ของเดือน
dd	แสดงวันที่
F	แสดงวันของสัปดาห์เป็นตัวเลข
EEE	แสดงวันของสัปดาห์เป็นตัวอักษรย่อภาษาอังกฤษ
a	แสดง AM,PM
HH	แสดงชั่วโมงแบบ 0-23
hh	แสดงชั่วโมงแบบ 1-12
kk	แสดงชั่วโมงแบบ 1-24
KK	แสดงชั่วโมงแบบ 0-11
mm	แสดงนาที
ss	แสดงวินาที



เมธอดในคลาส Date

คลาส **Date** เป็นเมธอดที่จัดการข้อมูลเกี่ยวกับ Date ที่สามารถให้ค่าวันที่และเวลาได้เช่นเดียวกับเมธอดในคลาส Calendar จะอยู่ในแพ็คเกจ java.util.Date มีรูปแบบ ดังนี้

```
Date DateName = new Date();
SimpleDateFormat df = new SimpleDateFormat("d-MM-yyyy KK:mm a");
System.out.println("Current DateTime is : " + df.format(DateName.getTime()));
```

> run DateTest

Current DateTime is : 23-07-2559 01:22 หลังเที่ยง

โดยที่

DateName	เป็นชื่อออบเจกต์
new Date()	เป็นการสร้างออบเจกต์ Date
getTime()	เป็นเมธอดที่ให้ค่าวันที่และเวลาปัจจุบัน